# Robotic Car Painting Cell Simulation

Technical Report – Robotics Simulation Intern Assessment

Yuktii AI Labs

Candidate: Ali Mohamed Elgohary | College: Faculty of Engineering | Email: alielgohary142005@gmail.com

## 1. Overview

This project simulates a simple robotic painting cell in which an industrial robot paints the exterior surfaces of a car body. The work covers the cell layout, robot and tool setup, Python-generated raster trajectories, collision checking, and an honest assessment of what the chosen robot can and cannot reach from a fixed position.

| Item | Value |
|---|---|
| Simulation software | RoboDK 6.1.1 (Free / Limited edition) with the RoboDK Python API |
| Robot selected | ABB IRB 6700-235/2.65 (reach 2.65 m, payload 235 kg) |
| End-effector | Generic Paint Sprayer (RoboDK tool library), 4 kg |
| Car model | Sedan body from GrabCAD (STEP), scaled and converted to STL |
| Scripting | Python 3 with the *robodk* package |

## 2. Software and Robot Selection

**Software.** RoboDK was chosen because it provides a large robot library, a Python API for generating programs automatically, and a built-in collision map. The Free edition was sufficient for simulation, although it does not include the paint (spray gun) particle simulation.

**Robot.** The ABB IRB 6700-235/2.65 is a large 6-axis industrial robot. Its 2.65 m reach and 235 kg payload comfortably cover a car-sized workpiece and a 4 kg spray gun. It is a general-purpose heavy robot rather than a dedicated paint robot, which is acceptable for a simulation exercise but is noted as a limitation in Section 8.

## 3. Cell Layout

The layout follows the reference drawing in the assignment. All dimensions are in millimetres.

| Element | Description |
|---|---|
| Floor | 12000 x 8000 x 100 box, top surface at Z = 0 (collision object) |
| Walls | Back wall 12000 x 100 x 4000 and side wall 100 x 8000 x 4000 (collision objects) |
| Robot base | At the station origin (0, 0, 0), no tilt or rotation |
| Car_Frame | Reference frame at Y = 2500 relative to the robot base; the car is mounted on it |
| Car body | Length 4800 along X, width about 1981 along Y, height about 1350 along Z; underside 150 mm above the floor; centred on Car_Frame |
| Clearance | About 1510 mm between the robot base and the near side of the car (2500 - 990) |

**Car scaling.** The reference car is 4800 x 1900 x 1600. The downloaded model, after scaling by a uniform factor of 2.655 (computed from the measured length), measures 4799.5 x 1981.5 x 1349.6. The length matches the reference, the width is within about 4%, and the height is about 250 mm lower, which is acceptable because the assignment specifies approximate dimensions.

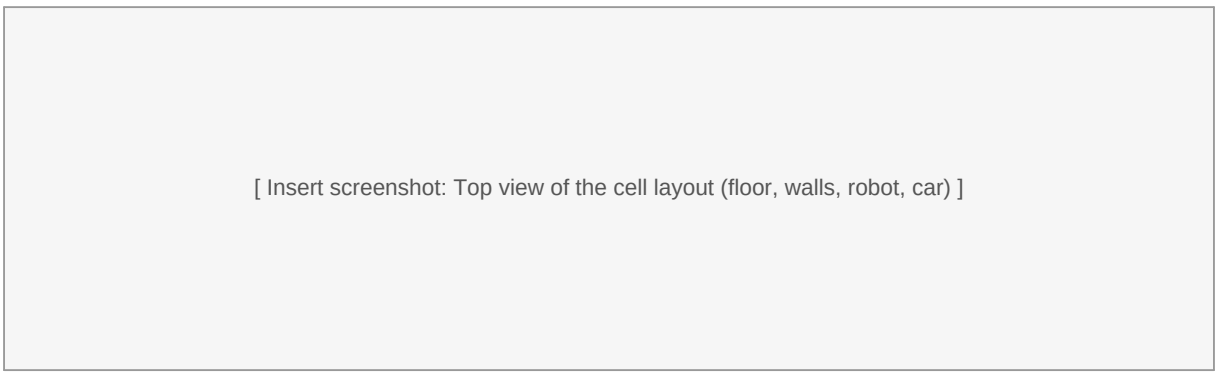


Figure 1. Top view of the cell layout (floor, walls, robot, car)

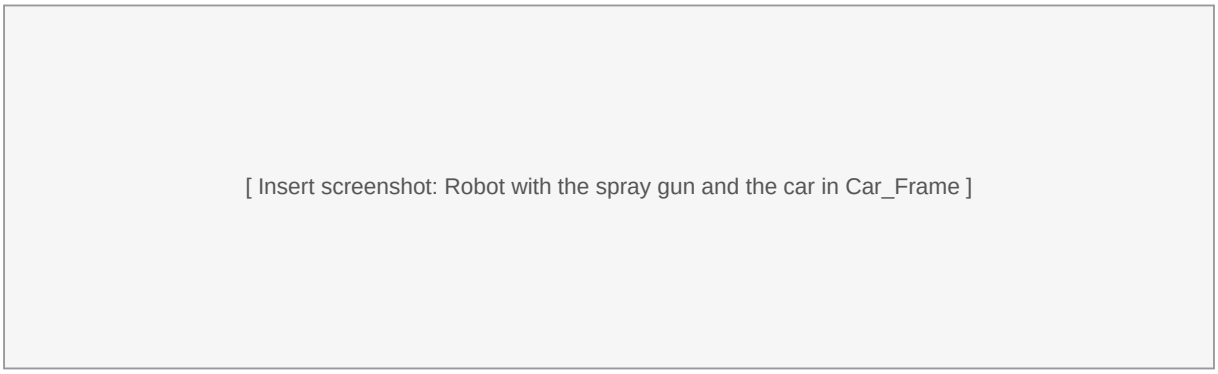


Figure 2. Robot with the spray gun and the car in Car_Frame

## 4. Approach

The cell was built step by step: layout, robot, tool, car model, trajectories, collision checking. Python scripts were used wherever possible because they are repeatable and make every number traceable:

- **Setup_Cell.py** creates the floor and walls from generated STL boxes.
- Measurement scripts export the scaled car to STL and read its bounding box to compute the scale factor.
- **Pose scripts** rotate the car 90° about Z so the length lies along X, centre it on Car_Frame, and set the ground clearance.
- **Paint_Side5.py, Paint_Roof2.py** generate the painting programs.
- **Make_Main.py** chains the programs into Main_Program.

**Tool and TCP.** The tool frame relative to the flange is X = 50, Y = 0, Z = 450 mm, with a 30° rotation about Y. The TCP is at the nozzle tip and the tool Z axis points toward the surface.

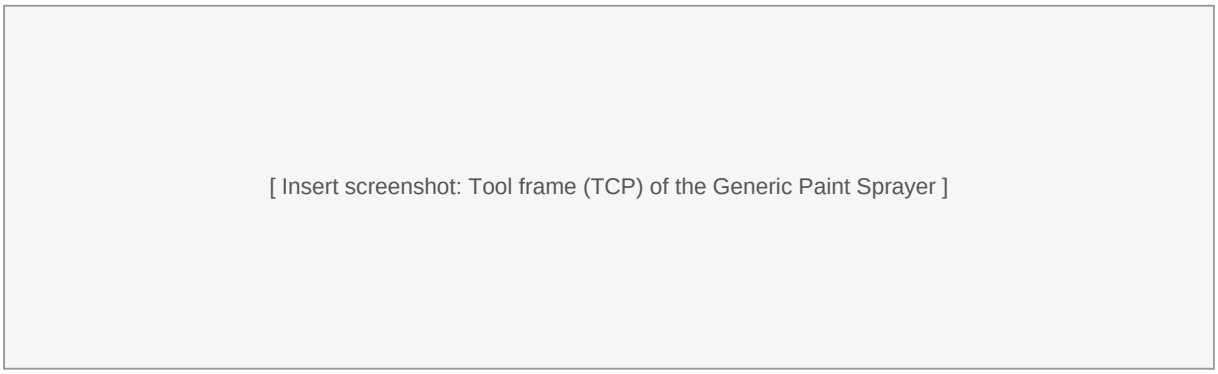


Figure 3. Tool frame (TCP) of the Generic Paint Sprayer

## 5. Trajectory / Path Planning

Each painting program is a **raster (zigzag) path** generated in Car_Frame coordinates. The script reads a known reachable starting pose of the robot, keeps its tool orientation, and shifts the position to produce parallel passes with fixed spacing.

| Program | Surface | Pattern | Notes |
|---|---|---|---|
| Paint_Side5 | Near side panel | 3 lines, +/-300 mm along X, 100 mm spacing | Gun at Y = -1200 (about 200 mm from the side at Y = -990) |
| Paint_Roof2 | Roof | 3 lines, +/-300 mm along X, 100 mm spacing | Gun raised to Z = 2000 to keep the forearm clear of the car |

**Program structure.** Each program starts with a safe approach point (pulled back from or raised above the surface), moves through the raster targets, and returns to the safe point.

**Singularity handling.** With linear moves, RoboDK reported *"Joint 5 crosses 0 degrees ... not allowed for a linear move"* (a wrist singularity). Tilting the tool did not remove it, so the raster points are connected with **joint moves (MoveJ)**. The trade-off is that the tool path between points is not perfectly straight, which is acceptable at this point spacing for a simulation.

[ Insert screenshot: Paint_Side5 raster path on the near side of the car ]

Figure 4. Paint_Side5 raster path on the near side of the car

[ Insert screenshot: Paint_Roof2 raster path on the roof ]

Figure 5. Paint_Roof2 raster path on the roof

## 6. Collision Checking

The RoboDK **Collision Map** was enabled for the robot links and the tool against the car body and both walls. Collision checking runs during simulation and stops the program with *"Collision detected. Robot stopped."*

| Issue found | Fix applied | Result |
|---|---|---|
| Collision at the first approach move (MoveJ) of the early side program | Added a safe approach point and used joint moves | Paint_Side5 runs to the end, no collision |
| Forearm touching the car when painting the roof at Z = 1700 | Raised the roof pass to Z = 1900-2000 and added a safe point above the roof | Paint_Roof2 runs to the end, no collision |

| Issue found | Fix applied | Result |
|---|---|---|
| Collisions flagged between neighbouring robot links (self-collision false alarms) | Reset the map with "Set default selection" so adjacent links are not checked against each other | Only robot-versus-car and robot-versus-wall checks remain |

**Note on diagnosis.** During debugging, some red highlighting turned out to be self-collision between adjacent links rather than contact with the car, so the Collision Map table was used to identify the actual colliding pairs before changing any program.

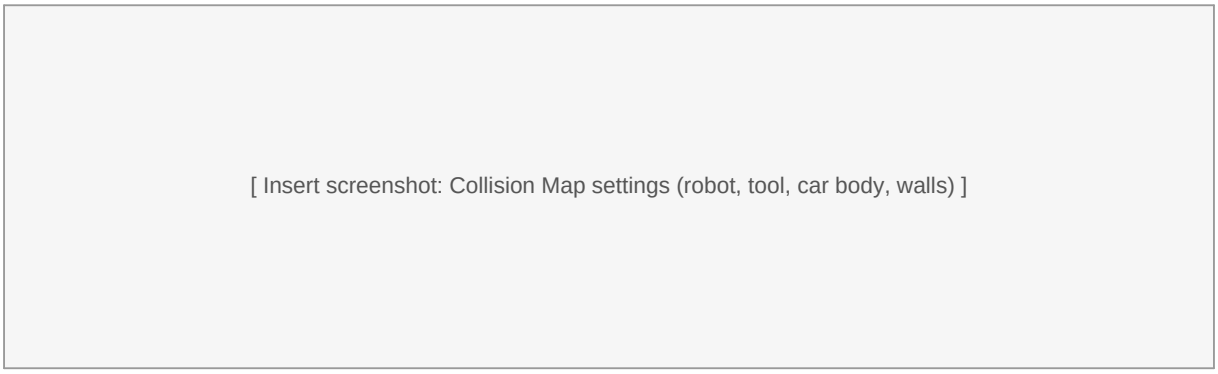


Figure 6. Collision Map settings (robot, tool, car body, walls)

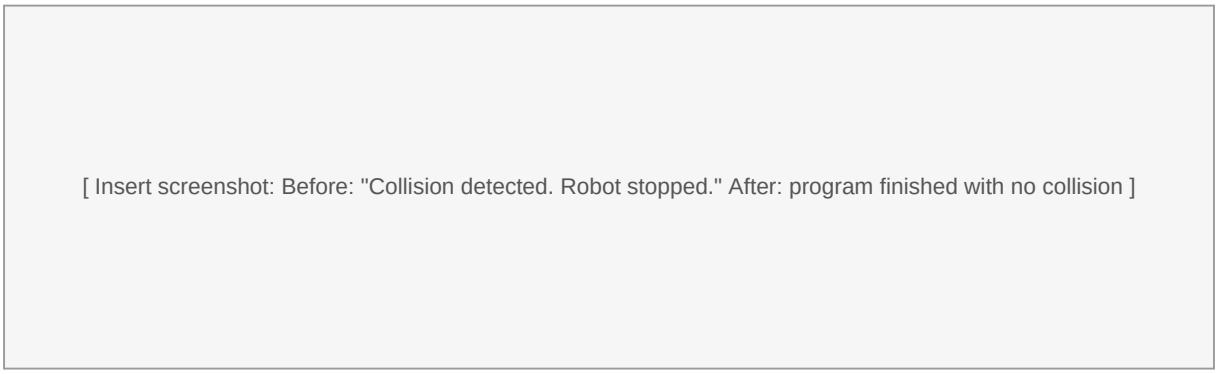


Figure 7. Before: "Collision detected. Robot stopped." After: program finished with no collision

## 7. Results

| Program | Status | Details |
|---|---|---|
| Paint_Side5 (near side) | Success | Finished without collision; path length 2826.6 mm, cycle time 4.6 s |
| Paint_Roof2 (roof) | Success | Finished without collision; path length 4630.0 mm, cycle time 8.9 s |
| Main_Program | Success | Runs Side5 then Roof2 in sequence |
| Hood, trunk | Not completed | Targets not reachable from the fixed robot position with the downward tool orientation (see Section 8) |

The simulation demonstrates the full workflow: layout, robot and tool, trajectory generation, collision checking, and a clear record of the problems met and how each was solved. The painted result is visualised by the raster path trace; the paint-particle simulation is not available in the Free edition.

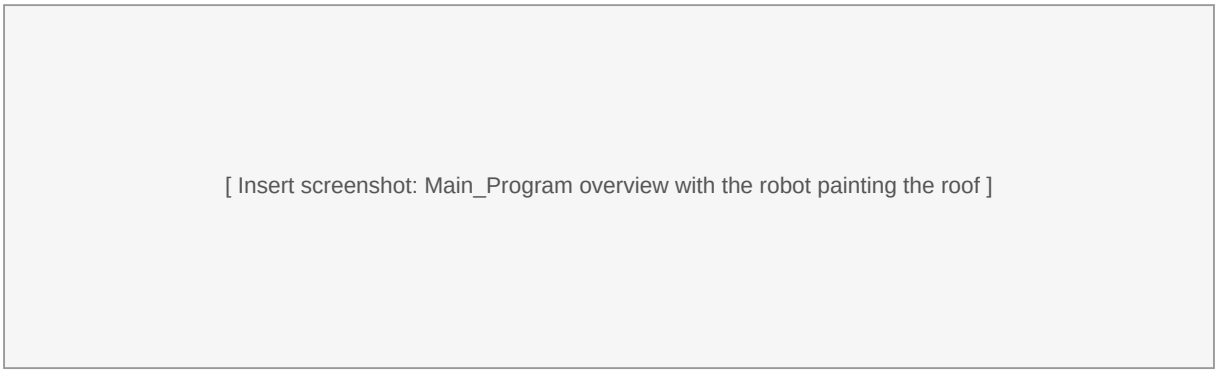


Figure 8. Main_Program overview with the robot painting the roof

## 8. Limitations and Reachability Analysis

With the robot at a single fixed position beside the car, not all surfaces can be painted:

- **Hood and trunk:** programs were generated at X = +/-1000 to 1500 mm, but RoboDK reported the first target as not reachable from Home. The surfaces sit at the edge of the 2.65 m reach, and a downward tool orientation reduces the usable reach further. These programs are not included in the working Main_Program.
- **Front, rear and far side:** the car is 4.8 m long and 2 m wide, so these surfaces are beyond the reach from one position, and the far side is also blocked by the car itself.
- **Robot choice:** a dedicated paint robot with a long, slim arm would be more suitable for the roof and hood than a heavy general-purpose robot.

**Proposed solutions (next steps):**

- Mount the robot on a **linear track** (7th axis) running along the car length.
- Place the car on a **turntable** so every side rotates into the robot's working area.
- Use RoboDK's collision-free motion planner and generate paths directly from the car surface instead of fixed raster planes.

## 9. Conclusion

A complete painting cell was built in RoboDK with an ABB IRB 6700-235/2.65, a paint sprayer, and a correctly scaled car model. The near side and roof are painted by Python-generated raster paths without collisions. Reachability limits for the hood, trunk, and other surfaces were identified and documented, and a linear track or turntable is the recommended extension.

**Deliverables:** simulation files (.rdk, Python scripts, car STL), screenshots, demo video, and the GitHub repository, all linked in the submission email.